

Transformer Architectures and How LLMs Work

Topics

- e! From RNNs to transformers: why attention changed everything
- e! Self-attention and multi-head attention
- e! Encoder-only, decoder-only, and encoder-decoder architectures
- e! Transformer model families: BERT, GPT, T5
- e! How an LLM is trained: pre-training, fine-tuning, and alignment (RLHF)
- e! Tokenization, context windows, and next-token prediction
- e! Efficiency innovations: GQA, Flash Attention, Mixture of Experts
- e! Small Language Models and when to use them

Learning Objectives

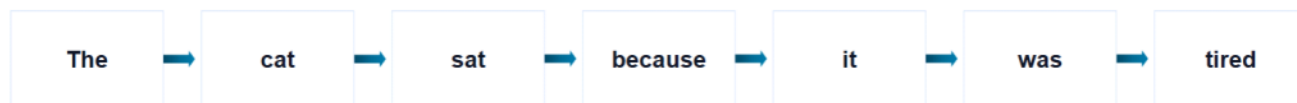
By the end of this lesson, you will be able to:

- e! Explain how transformers overcome the limitations of RNNs.
- e! Describe self-attention, multi-head attention, and QKV processing.
- e! Compare encoder-only, decoder-only, and encoder-decoder architectures.
- e! Distinguish BERT, GPT, and T5 based on their use cases.
- e! Outline LLM training from pre-training to fine-tuning and RLHF.
- e! Select efficient models using context, cost, latency, and task needs.

From RNNs to Transformers

The RNN Bottleneck

Long dependency must travel through every intermediate step



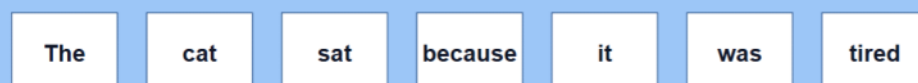
RNN issue

- Sequential processing slows training.
- Earlier context can fade across many tokens.
- Parallelism is limited because each step depends on the last.

Key idea: Transformers remove this bottleneck by letting every token compare itself with every other token in the same layer.

Why Attention Changed Everything

Attention lets the model decide which tokens matter most for each token



Token "it" gives more weight to "cat" than to nearby but less relevant words.

Global context

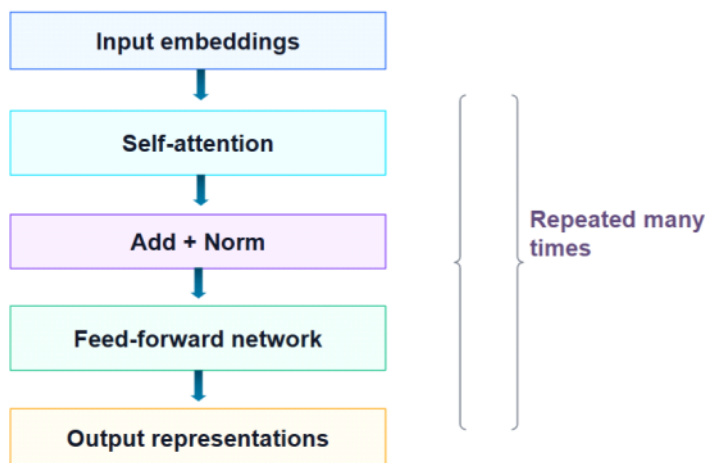
Parallel training

Long-range links

The attention score is learned from data. It is not a fixed rule written by a developer.

Transformer Block: The New Basic Unit

A transformer stacks repeated blocks that mix token context, transform features, and preserve useful signals



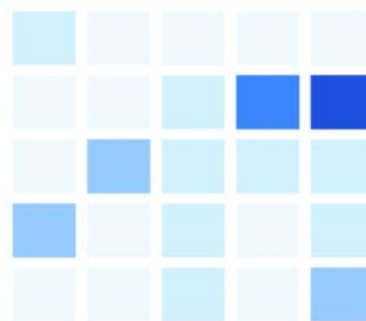
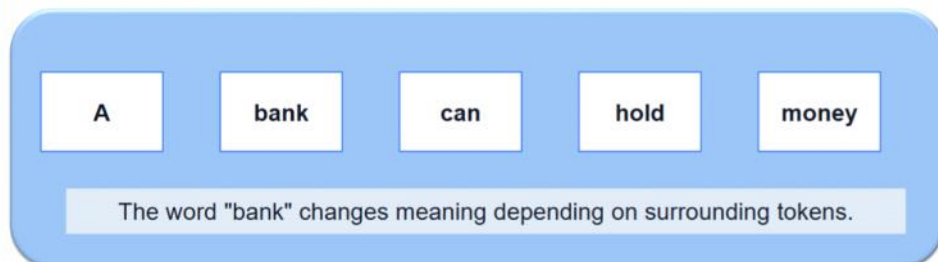
- Attention mixes information across tokens.
- Feed-forward layers transform each token representation.
- Residual connections make deep stacks trainable.
- Layer normalization stabilizes values across layers.

Mental model: a transformer is a repeated pattern of "look around, update meaning, keep useful signal."

Self-Attention and Multi-Head Attention

Self-Attention: Tokens Compare with Tokens

Each token asks: which other tokens should influence my current meaning?

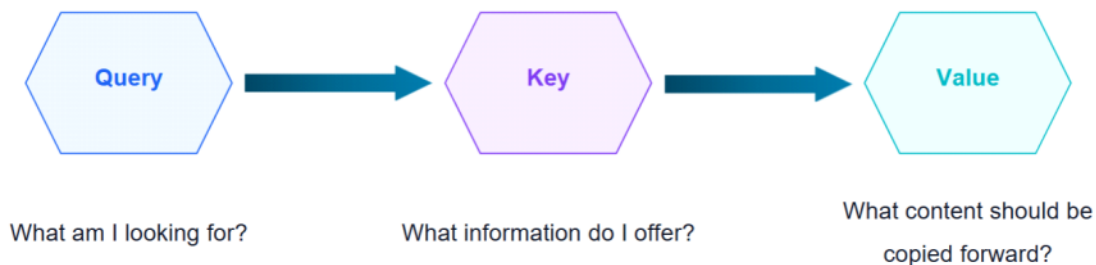


Attention heatmap

- Rows: token updated.
- Columns: tokens considered.
- Darker cells: stronger influence.

Query, Key, Value: The Attention Triplet

Attention uses three learned views of each token representation

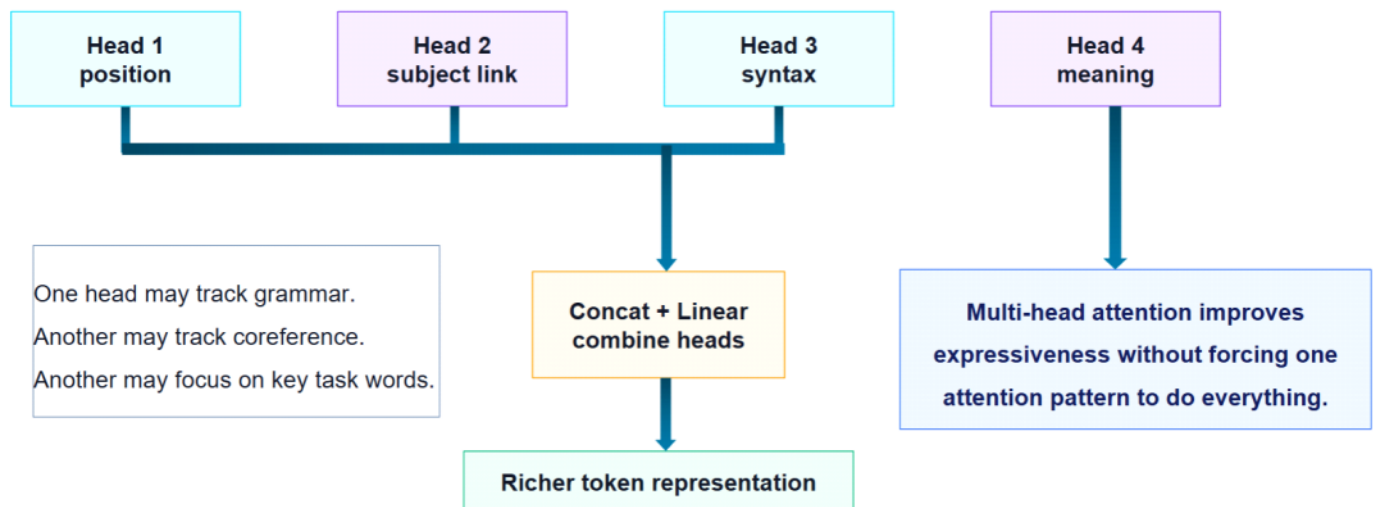


Simplified scoring

$\text{score}(\text{token}_i, \text{token}_j) = \text{Query}_i \text{ dot } \text{Key}_j \rightarrow \text{weighted sum of Values}$

Multi-Head Attention: Many Views at Once

Different heads can specialize in different relationships in the same sentence.



Demonstration 1 : Visualizing Self-Attention in a Transformer Model

Encoder-only, Decoder-only, and Encoder-Decoder Architectures

Three Transformer Architecture Patterns

Encoder-only

Reads full input at once

Best for understanding



Input
all visible

Decoder-only

Generates left to right

Best for text generation



Next token
masked future

Encoder-decoder

Reads input, then generates
output

Best for transformation tasks



Input encoder
+ output decoder

Selection rule: Match the architecture to whether the task is understanding, generation, or transformation.

Encoder-Only: Understand the Whole Input

Encoder models use bidirectional attention, so every token can attend to both left and right context.

[CLS]

The

movie

was

great

Bidirectional attention: all tokens can share context



Classification
Sentiment: Positive

- Good for classification and extraction.
- Uses both previous and following words.
- Often represented by BERT-style models.

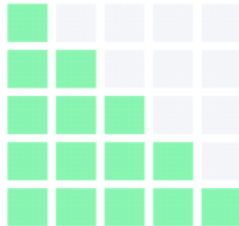
Not ideal for open-ended generation because it is trained to understand full input, not predict a long reply one token at a time.

Decoder-Only: Generate One Token at a Time

Decoder-only models use causal masking: a token can look left, but not into the future.



- Good for chat, writing, coding, reasoning, and agents.
- Predicts the next token repeatedly until a stop condition.
- Commonly represented by GPT-style models.



Causal mask: only earlier tokens are visible

Generation is just repeated next-token prediction under a prompt, system instructions, and conversation history.

Architecture Choice by Task

Use architecture as a practical design decision, not just a theory label.

Task type	Best fit	Reason
Sentiment classification	Encoder-only	needs full input understanding
Chat assistant	Decoder-only	generates flexible responses
Translation	Encoder-decoder	maps source to target text
Keyword extraction	Encoder-only	finds labels/spans in input
Code generation	Decoder-only	predicts structured sequences

Remember: modern products often combine multiple models.
Example: encoder for retrieval/ranking plus decoder for final response.

Demonstration 2 : Comparing Encoder-Only, Decoder-Only, and Encoder-Decoder Models

Transformer Model Families

BERT, GPT, and T5 at a Glance

Model families differ in architecture, training objective, and strongest use cases.

BERT

Encoder-only

- **Training focus**
Masked language modeling

- **Common use**
Classification, search ranking, extraction

GPT

Decoder-only

- **Training focus**
Next-token prediction

- **Common use**
Chat, coding, writing, agents

T5

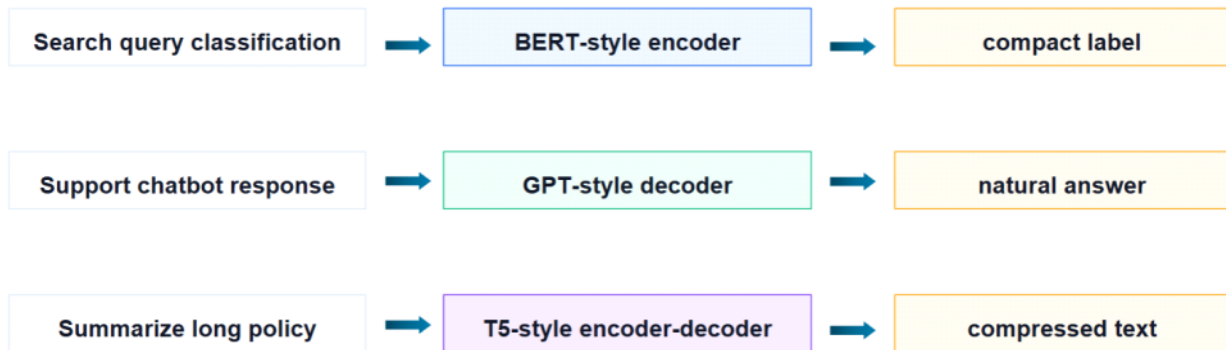
Encoder-decoder

- **Training focus**
Text-to-text objective

- **Common use**
Translation, summarization, transformations

Choosing a Family for Real Work

The best model family depends on the job the application must perform



In modern systems, these roles can be mixed: an encoder retrieves relevant context; a decoder produces the final user-facing answer.

Model Family Is Only the Starting Point

Architecture matters, but deployment choices also depend on size, data, alignment, and latency.

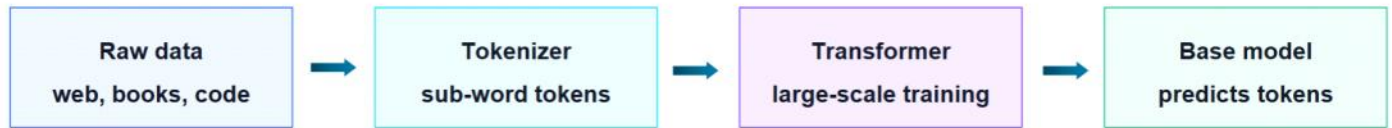
Architecture	encoder / decoder / both
Training data	domain coverage and quality
Model size	parameters and memory
Alignment	instruction following and safety
Serving setup	latency, cost, throughput

- A small aligned model can outperform a larger base model on a narrow task.
- A decoder-only model may still need retrieval or tools for fresh facts.
- Model choice should be validated with tests, not just architecture labels.

How an LLM is Trained

Pre-Training: Learn Patterns from Massive Text

The base model learns language, facts, style, syntax, and reasoning traces by predicting missing or next tokens



- Objective is usually simple: predict the next token or masked tokens.
- Scale comes from huge datasets, many parameters, and long training runs.
- The result is powerful but not necessarily instruction-following or safe.

Fine-Tuning: Adapt the Base Model to a Purpose

Fine-tuning teaches the model a narrower skill, style, domain, or instruction-following behavior



Examples teach the model how to respond, not just how to predict text continuation.

```
Instruction tuning data

# training pair idea
{
  "instruction": "Summarize this ticket",
  "input": "Customer asks for refund...",
  "output": "intent=refund; priority=high"
}
```

- Domain tuning improves vocabulary and patterns.
- Instruction tuning improves task-following behavior.
- Parameter-efficient methods can reduce training cost.

Alignment and RLHF: Shape Helpful Behavior

Alignment reduces harmful, unhelpful, or off-policy behavior after the model has learned language.



- The goal is not only correctness; it also includes tone, safety, refusal behavior, and instruction following.
- Human preference data teaches which answer is better when several answers are possible.
- Alignment is an ongoing process because real user behavior exposes new edge cases.

Tokenization, Context Windows, and Next-Token Prediction

Tokenization: Text Becomes Sub-Word Pieces

A tokenizer converts text into token IDs that the model can process



```
Tokenizer example

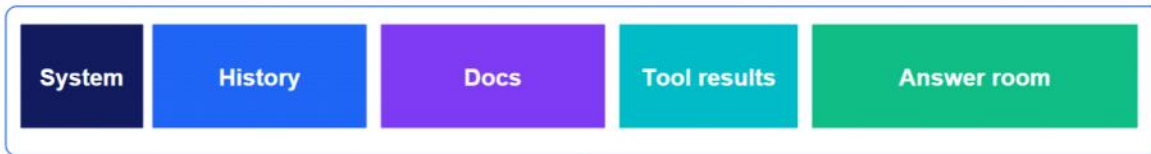
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
ids = tok.encode("unbelievable")
print(ids)
print(tok.convert_ids_to_tokens(ids))
```

- One word can become many tokens.
- Spaces, capitalization, and rare words affect token count.
- Token count controls context length, speed, and cost.

Context Window: One Shared Token Budget

System prompt, user messages, retrieved documents, tool results, and the answer all compete for the same window.



↓
Overflow gets truncated or forces shorter answers

- Long prompts leave less room for output.
- Tool traces and chat history can silently crowd out key facts.
- Chunking, summarization, and retrieval help keep relevant tokens in view.

Next-Token Prediction: The Generation Loop

A decoder model predicts one token, appends it, then repeats until it stops.



Pseudo-code

```
prompt = "The capital of France is"

while not stop:
    logits = model(prompt)
    next_token = sample(logits)
    prompt += next_token
```

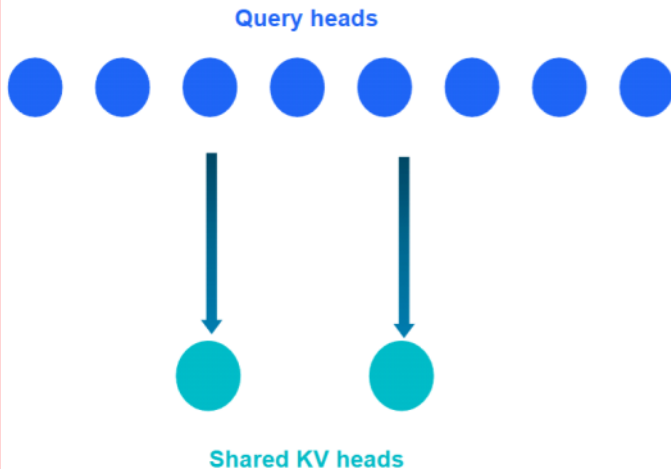
- Temperature changes how random token selection is.
- max_tokens limits how long generation can continue.
- Stop sequences tell the model when to end output.

Demonstration 3 : Tokenizing Text and Exploring Context Window Limits

Efficiency Innovations

GQA: Share Key-Value Heads to Save Memory

Grouped-query attention reduces inference memory by letting multiple query heads share fewer key-value heads

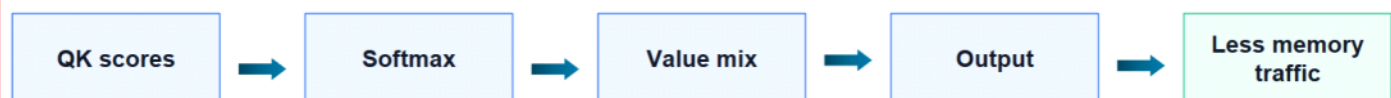


- Faster decoding for long conversations.
- Lower KV-cache memory during inference.
- Quality is usually close to full multi-head attention for many models.

Why it matters: attention can become expensive at serving time because every generated token reuses cached keys and values.

FlashAttention: Same Math, Smarter Memory Use

FlashAttention speeds attention by reducing slow memory movement on GPUs.



Standard attention can materialize large score matrices; FlashAttention streams blocks through fast memory.

- Especially useful for long context and large batches.
- Improves speed without changing the model objective.
- Requires optimized kernels, not a different prompt or training dataset.

Mixture of Experts: Activate Only Part of the Model

MoE models contain many expert networks, but route each token to only a small subset



Small Language Models and When to Use Them

Small Language Models: Not Just Smaller LLMs

SLMs are optimized for practical tasks where speed, cost, and control may matter more than broad reasoning.

- Faster responses on common tasks.
- Lower serving cost and memory requirements.
- Easier to deploy on controlled infrastructure.
- Can be fine-tuned for narrow domain behavior.

Trade-off: SLMs may struggle with complex reasoning, broad knowledge, or unusual edge cases compared with larger models.



When to Use an SLM

Choose a small model when the task is well-scoped and the operating constraints are tight.

Use SLM

classification, extraction, routing, FAQ, template responses

Use Larger LLM

multi-step reasoning, open-ended chat, code planning, complex analysis

Hybrid

small router + large reasoner + retrieval/tools

Common production pattern: start with a small model for cheap routing. Escalate only difficult cases to a larger model.

Model Selection Framework

A model choice should pass task, quality, cost, and latency constraints together.

Task complexity

simple / narrow / complex

Cost budget

per call and total volume

Quality bar

accuracy and safety threshold

Data/privacy

cloud, private, or edge

Latency target

interactive or batch

Summary

In this lesson, you have learned to:

- ! Understand how attention enables parallel and context-aware processing.
- ! Identify major transformer architectures and model families.
- ! Explain tokenization, context windows, and next-token generation.
- ! Describe pre-training, fine-tuning, and alignment stages.
- ! Recognize efficiency methods such as GQA, FlashAttention, and MoE.
- ! Choose between small and large language models for different tasks.